

RAPPORT D'AUDIT

Audit avant / après corrections

Sécurité, fiabilité, supervision, interface et exploitation. Ce rapport explique les défauts observés, les corrections appliquées, les commandes de vérification et les preuves de validation.

Edition 2026

Document de référence - aucune valeur secrète n'est incluse.

Sommaire

Sommaire	2
1. Objet et périmètre	3
2. Architecture auditée	3
Architecture des équipements et services	4
Lecture logique complémentaire	5
3. Situation initiale	6
4. Corrections appliquées	6
5. Chaîne de supervision validée	7
6. Commandes de contrôle	7
Application et conteneur	7
Prometheus et Alertmanager	7
Sauvegardes	7
7. Preuves de validation	8
8. État après audit	8
9. Risques résiduels et priorités	8

1. Objet et périmètre

L'audit couvre l'application Flask, Docker, Nginx, Cloudflare Tunnel, Prometheus, Grafana, Alertmanager, Emma_IA, les sauvegardes et le poste Windows supervisé. Il distingue les observations initiales des preuves obtenues après correction.

Limite : il s'agit d'un audit technique du projet et de sa configuration, pas d'un test d'intrusion externe certifié.

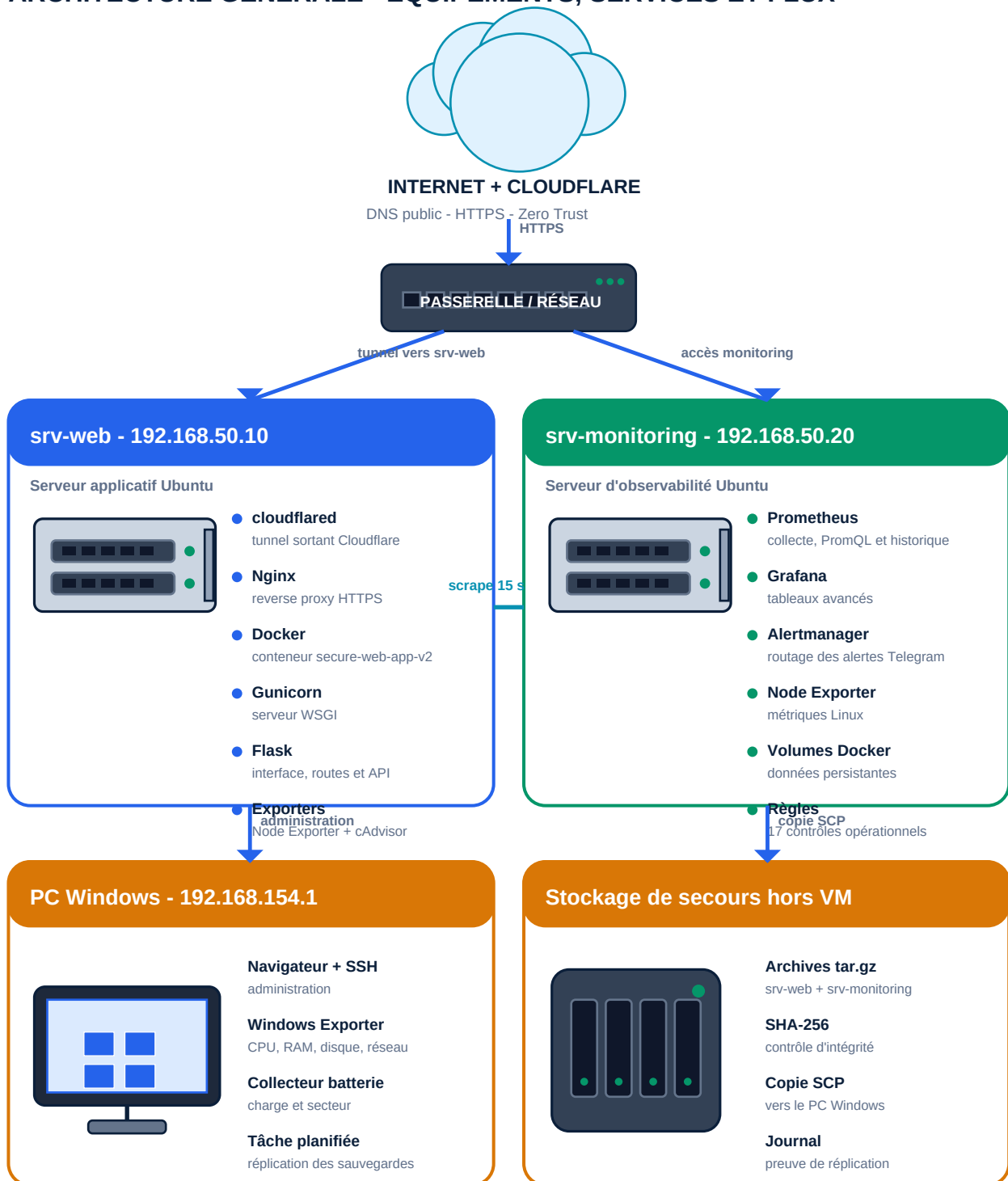
2. Architecture auditée

Le périmètre ne se limite pas au code Flask. Il comprend les équipements physiques ou virtuels, les réseaux, le chemin HTTPS, la collecte des métriques, les volumes et la copie de secours hors VM.

Le schéma suivant localise chaque composant sur son équipement réel et distingue la production, l'observabilité, l'administration et la sauvegarde hors VM.

Architecture des équipements et services

ARCHITECTURE GÉNÉRALE - ÉQUIPEMENTS, SERVICES ET FLUX

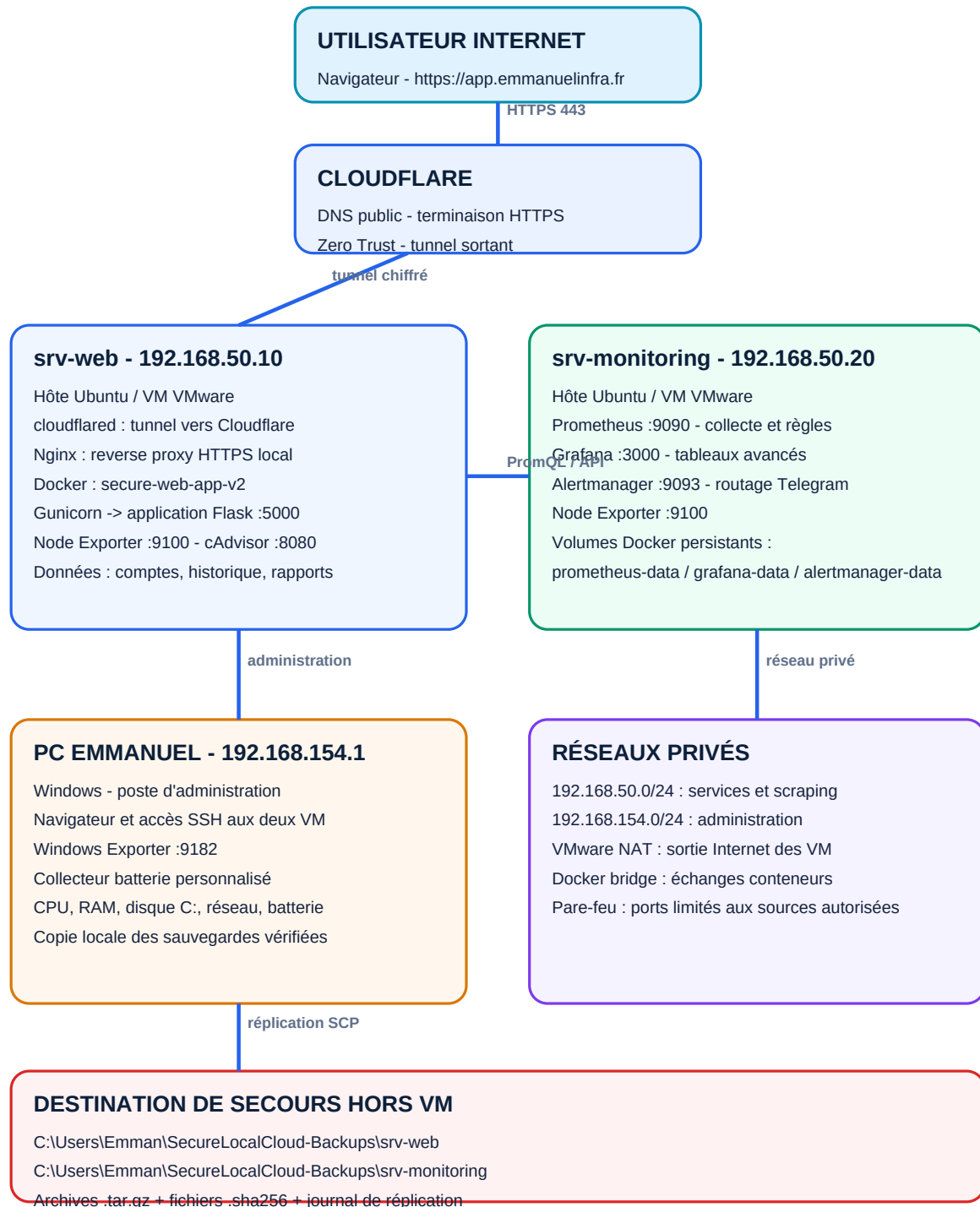


Réseau services : 192.168.50.0/24 | Réseau administration : 192.168.154.0/24

Chaque technologie est placée sur l'équipement où elle s'exécute réellement.

Lecture logique complémentaire

ARCHITECTURE PHYSIQUE ET EMPLACEMENT DES SERVICES



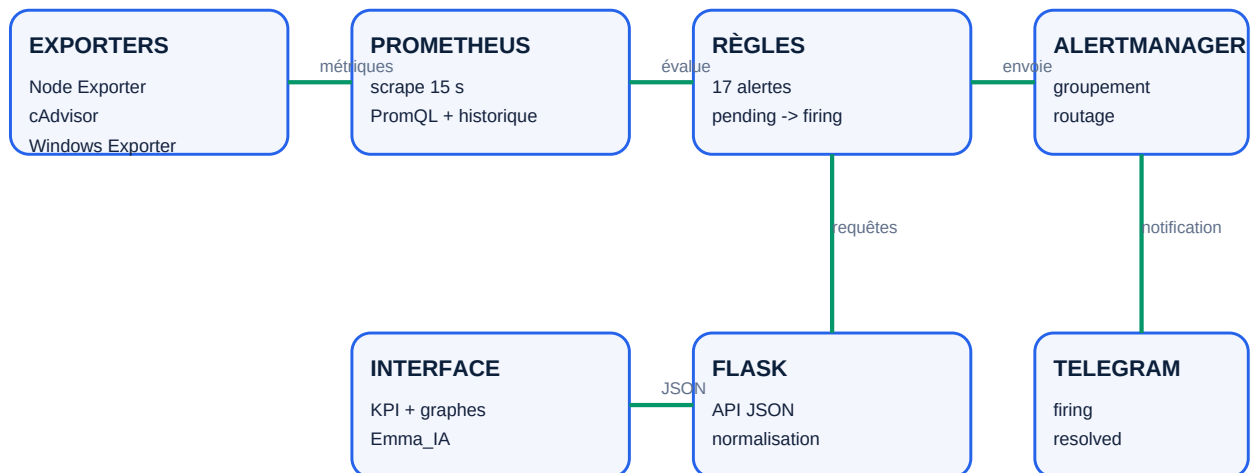
3. Situation initiale

Domaine	Avant correction	Risque
Emma_IA	Intentions trop génériques, réponses répétées, confusion entre documentation et temps réel.	Décisions fondées sur une réponse inadaptée.
Monitoring	Historique figé, heures UTC affichées comme locales, KPI parfois vides.	Lecture opérationnelle trompeuse.
Multi-équipement	Seul srv-web était réellement présenté.	srv-monitoring et le PC restaient invisibles.
Alertes	Règles centrées sur Linux et couverture Windows incomplète.	Incident PC ou batterie non signalé.
Sauvegardes	Archives locales sans réplication PC automatisée.	Perte simultanée de la VM et de ses sauvegardes.
Secrets	Permissions et exclusions Git à renforcer.	Publication ou lecture accidentelle.
PDF	Texte de commandes peu contrasté et échappement incomplet.	Rapport illisible ou contenu injecté.
Interface	Doublons de navigation, mobile encombré, impression confuse.	Expérience peu crédible et difficile à utiliser.

4. Corrections appliquées

Correction	Résultat vérifié
Routage Emma_IA multi-équipement	État ciblé, comparaison, équipement prioritaire, batterie et volumes persistants.
API et PromQL configurables	srv-web, srv-monitoring et PC Windows interrogés par labels et instances configurables.
Interface multi-équipement	Vue globale et onglets dédiés, KPI, historiques, services et stockage réel.
Collecteurs	Node Exporter sur les deux serveurs, cAdvisor, Windows Exporter, batterie et volumes Docker.
Alertes Telegram	17 règles chargées ; alerte RAM Windows reçue et résolution prise en charge.
Sauvegarde + réplication	Archives serveur, empreintes SHA-256, copie automatique vérifiée sur Windows.
Durcissement	.env en 600, credentials en 600, exclusions Git, Emma_IA en lecture seule.
Qualité	Tests Python, tests interface, validation Docker Compose et contrôles HTTP.

5. Chaîne de supervision validée



Principe essentiel : une métrique décrit un état chiffré ; une règle décide si cet état devient une alerte.

L'interface lit Prometheus mais ne remplace ni le stockage temporel ni le moteur d'alertes.

Les corrections ont été vérifiées à chaque niveau : exposition des exporters, targets Prometheus UP, requêtes PromQL, API Flask, graphiques, règles firing, réception Alertmanager et notification Telegram.

6. Commandes de contrôle

Application et conteneur

```

docker compose config --quiet
docker compose ps web
docker inspect secure-web-app-v2 --format 'État={{.State.Status}} Santé={{.State.Health.Status}}'
curl -k -I https://127.0.0.1
docker logs --tail 50 secure-web-app-v2

```

Prometheus et Alertmanager

```

promtool check rules /etc/prometheus/alerts.yml
amtool check-config /etc/alertmanager/alertmanager.yml
curl -fsS http://127.0.0.1:9090/api/v1/targets
curl -fsS http://127.0.0.1:9090/api/v1/alerts
curl -fsS http://127.0.0.1:9093/api/v2/alerts

```

Sauvegardes

```

systemctl list-timers --all | grep backup
sha256sum -c ./*.tar.gz.sha256
# Windows
Get-ScheduledTaskInfo -TaskName 'Secure Local Cloud - Replication'
Get-Content $env:USERPROFILE\SecureLocalCloud-Backups\logs\replication.log -Tail 30

```

7. Preuves de validation

Preuve	Résultat
Emma_IA Docker	26 tests réussis.
Interface locale	17 tests visuels réussis.
Syntaxe Python	Compilation réussie.
Documentation	0 lien relatif cassé.
Recherche de secrets	Aucun motif sensible détecté dans le dépôt ni son historique local.
Production	Conteneur running et healthy ; HTTPS local et URL publique répondent par redirection d'authentification.
Telegram	Alerte WindowsMemoryCritical reçue par l'administrateur.
Métriques	Trois équipements UP ; historiques et volumes persistants visibles.

8. État après audit

La plateforme est exploitable comme laboratoire multi-équipement : les données temps réel sont distinguées de la documentation, l'interface affiche les trois sources, les alertes couvrent Linux et Windows, et les sauvegardes possèdent une copie hors VM.

Les améliorations ne suppriment pas le besoin de maintenance. Les dépendances, images, certificats, règles d'alerte et restaurations doivent être revérifiés périodiquement.

9. Risques résiduels et priorités

- Faire tourner tout secret ayant été exposé et reconstruire les exports publics avant publication.
- Tester une restauration complète sur une machine isolée au moins avant chaque migration.
- Épingler les images Docker par version ou digest au lieu d'utiliser latest.
- Ajouter une CI GitHub pour les tests, le scan de secrets et la validation des configurations.
- Prévoir une seconde destination de sauvegarde chiffrée hors du PC principal.